

Efficient Hardware Design of Parameterized Posit Multiplier and Posit Adder

Sadhu Sai Ram
Electronic Systems Engineering
Indian Institute of Science
Bengaluru, India
sadhuram@iisc.ac.in

Kuruvilla Varghese, Senior Member, IEEE
Electronic Systems Engineering
Indian Institute of Science
Bengaluru, India
kuru@iisc.ac.in

Abstract—Posit arithmetic, an alternative to IEEE 754 floating-point arithmetic, offers higher precision and a wider dynamic range. In posit, the number of bits allocated to fractional part varies based on the magnitude of the number. For representing very large/small numbers, the exponent field uses more bits and fractional field uses less bits. The run-time variations in posit number fields inherit an interesting hardware design challenge for posit arithmetic architectures. The word size (N) and exponent size (es) defines posit format. For the same word size, it is possible to have different dynamic range and accuracy based on exponent size (es) which is absent in floating-point. This work proposes optimized parameterized hardware implementation of posit operators, namely multiplier and adder along with its pipelined architectures in Verilog HDL. Implementation results on Virtex-7 (xc7vx330t-3ffg1157) FPGA for posit multiplier and posit adder are found to be better than the existing designs. Non-pipelined posit multiplier's datapath delay reduced by 21% and the number of LUTs are reduced by 9%. Non-pipelined posit adder's datapath delay reduced by 4% and the number of LUTs are reduced by 15%. Software implementation of the posit operators in MATLAB is carried out for validation. There is a 100% match between software and hardware results.

Index Terms—Posit, FPGA, LUTs, Decoder, Encoder, Regime, Overall Exponent, Fractional part

I. INTRODUCTION

Floating-point arithmetic provides precision and range necessary for handling real-world numerical computations across multiple disciplines such as scientific computing, machine learning etc. However, representing real numbers with higher precision and higher dynamic range improves accuracy of various applications. This can be achieved using Posit (Type-3 Unum), a new number system similar to floating-point (FP) number system [1]. It is proposed by J. L. Gustafson [2]. Posit's non-uniformed number distribution fits well with data distribution of deep neural networks. Posit numbers are represented in 2's complement format. The format of posit is $\langle N, es \rangle$, where N is total number of bits and es is the exponent size.

Posit format : $\overbrace{s \text{ } rrrr \dots rrr \text{ } e_1 e_2 e_3 e_4 \dots e_{es} \text{ } f_1 f_2 f_3 f_4 \dots}^{N \text{ bits}}$
sign bit regime bits exponent bits, if any fraction bits, if any

Decimal value of posit [2]: $(-1)^s * \text{used}^k * 2^{Exp} * f$
Decimal value of posit [2]: $(-1)^s * 2^{OE} * f$, where s: sign,

used = $2^{2^{es}}$, Fractional f: $1. f_1 f_2 f_3 \dots f_p$, k: related to run length of identical bits in regime, **Exp**: exponent value, **Overall exponent(OE)**: $k * 2^{es} + \text{Exp}$. OE is defined as concatenation of Regime and exponent-bits (with maximum size of es bits). The fields present in posit are namely Sign, Regime, Exponent and Fractional. In FP format, the number of bits allocated is fixed to different fields. In posit, bit allocation to different fields varies. Due to this posit provide better accuracy for real numbers of values around 1 and less accurate for very large/small numbers. The following combination of fields can exist, (Sign + Regime) or (Sign + Regime + Exp) or (Sign + Regime + Exp + Fractional). In posit, there will be no overflow or underflow. Instead, it saturates at maxpos (used^{N-2}) or minpos ($\text{used}^{-(N-2)}$).

This work proposes optimized hardware for parameterized posit multiplier and posit adder (pipelined and non-pipelined) making it suitable for high speed applications. Decoding and encoding algorithm's hardware implementation have been optimized in terms of datapath delay and FPGA lookup tables (LUTs). Software implementation of posit operators in Matlab for verification of hardware results is also done.

The organisation of the paper is as follows: Section II discusses the related works. Section III discusses designs of posit decoder, posit encoder, posit multiplier and posit adder. Section IV discusses the evaluation of posit adder and multiplier. Section V describes implementation results. Finally, conclusion and future works are discussed in Section VI.

II. RELATED WORKS

First implementation of posit arithmetic operators in RTL has been proposed by Manish kumar Jaiswal [3]. Later, PACoGen, an improved version of previous implementation is proposed by the same group [4]. In the same work, pipelined $\langle 32, 6 \rangle$ posit adder and multiplier are also proposed. Junjie Hou [7] has proposed pipelined $\langle 32, 3 \rangle$ posit adder and multiplier. Very few implementations of pipelined architectures for posit are present in literature. R. Chaurasiya [6] has proposed parameterized posit arithmetic hardware generator. Recently, Lakshmi Bhanuprakash Reddy K [9] has proposed energy efficient and low delay posit multiplier.

III. DESIGN

A. Posit decoder with unsigned fractional part output

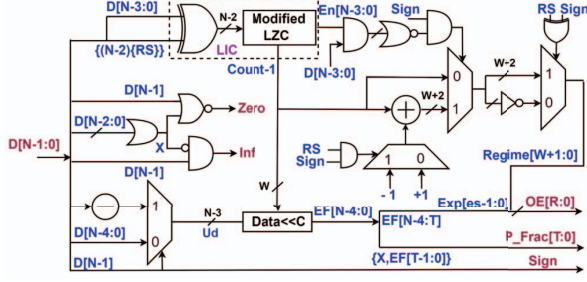


Fig. 1. Proposed architecture of posit decoder (outputs: unsigned fractional part, Sign, OE, Zero, Inf). N: Operand width, OE: {Regime, Exp}, T: $N-es-3$, Regime width (R): $W+1+es$, W: $\log_2(N)$, Regime Sign (RS): $D[N-2]$

Decoder design with unsigned fractional part [4] as output can be useful for unsigned multiplication and division implementation using non-restoring algorithm. Decoding of posit number needs to be done on unsigned number [2]. As the input posit data is in 2's complement form, it needs to be converted to unsigned format for decoding. Later, it is given to leading identical bits counter (LIC) for counting number of identical bits. The identical bits along with termination bit in unsigned data (**Ud**) are left shifted by count of LIC to form **EF**[$N-4:0$]. Initial **es** bits in **EF** will be Exponent (Exp) part and remaining bits will be fractional part. Special cases (Zero, Inf) processing also needs to be done. Using count from LIC, regime computation is performed as mentioned below.

Regime $\leftarrow$ $Ud[N-2] ? \text{count} - 1 : -\text{count}$.

By skipping start bit of the sequence to LIC [8], the LIC gives count - 1 as output. Hence, the regime computation can be simplified to **Regime** $\leftarrow$ $Ud[N-2] ? \text{count} : \sim \text{count}$.

In the proposed design shown in Fig. 1, the input data is directly given to LIC without converting it to unsigned format (datapath delay of 2's complement and the multiplexer is eliminated). Since, positive data in 2's complement format and unsigned format is same, there will be no error in count of LIC. There will be no error in LIC count for negative posit data, provided **EF**[$N-4:0$] is non-zero. Error will be present in LIC count for negative posit data, if **EF** is zero. There are two possible scenarios of posit data when **EF** can be zero, either identical bits are ones (e.g. 8'b11110000) or zeros (e.g. 8'b10010000), terminated by opposite bit. Unsigned format of the above numbers are 8'b00010000 and 8'b01110000.

Errors in count of LIC can be detected and corrected, once **EF** is available. But, **EF** is available after left shifting of **Ud**. The corrections in count of LIC and regime computation's datapath delay will be added instead of unsigned conversion datapath delay. The added datapath delay can be removed by below methodology. Apart from count of identical bits, LIC will also give **En**[$N-4:0$] as output. **En** comprises of zeros until identical bits followed by termination bit are present and remaining bits are filled with ones. Later, **En** is 'ANDed' with input data (**D**) which makes it equal to **EF** when error

happens. As a result, the corrections in count of LIC and regime computation can be done in parallel to left shifting of **Ud** to get **EF**. Hence, overall datapath delay reduced by removing the unsigned conversion block. Xor of **sign** and **RS** will decide whether it is positive regime or negative regime. Extracted Exp from **EF** is combined with regime to form **OE**.

B. Posit decoder with 2's complement fractional part output

Decoder design shown in Fig. 2 with 2's complement fractional part as output can be useful for additions. In this design, posit data is given directly to LIC similar to the design in Section III.A. As the fractional data present in posit input is in 2's complement form, there is no need of converting posit data to unsigned form. This saves resources and reduces datapath delay. The errors in count of LIC are corrected using **Exp_Frac**. Extracted Exp from **Exp_Frac** needs to be converted to unsigned form as it is later concatenated with regime to form **OE**. Extracted Exp is 'XORed' with sign to get unsigned form provided **Exp_Frac**[$T-1:0$] is non-zero. The corrections in extracted Exp and integer part are done when **Exp_Frac**[$T-1:0$] is zero for negative posit data as shown in Fig. 2. Extra resources are required for applying corrections. However, the datapath delay caused by corrections are minimal compared to unsigned conversion block and is independent of data width. The added datapath delay cannot be overlapped with left barrel shifter like in previous design as correction in Exp can be carried out once **Exp_Frac** is available. As a result, corrections in count of LIC is done after **Exp_Frac** is available.

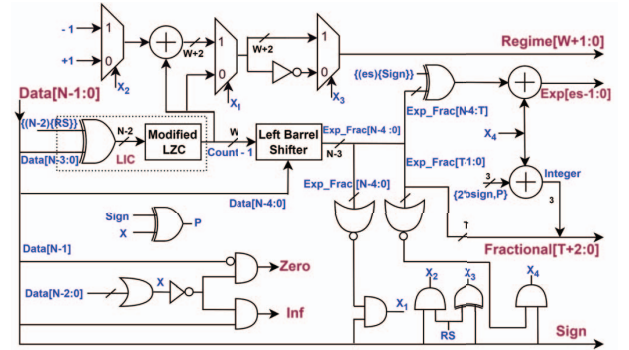


Fig. 2. Proposed architecture of posit decoder (outputs: 2's complement fractional part, Sign, OE, Zero, Inf). N: Operand width, W: $\log_2(N)$, Regime Sign (RS): $D[N-2]$, R: $W+1+es$, OE[R:0]: {Regime, Exp}, T: $N-es-3$

C. Posit encoder

Posit encoder accepts unsigned fractional part, overall exponent (OE), sign, special cases (Inf, Zero) and gives posit data as output. Posit encoder is divided into two parts, namely posit encoder(a) and posit encoder(b) as shown in Fig. 3 and 4 respectively. In posit encoder(a), **OE** along with special cases are processed to get run length shift and preamble **Reg_Exp** (identical bit, termination bit, Exp). In posit encoder(b),

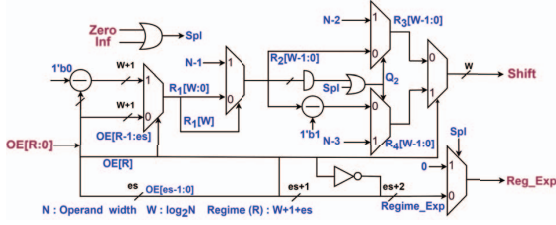


Fig. 3. Proposed architecture of posit encoder(a)

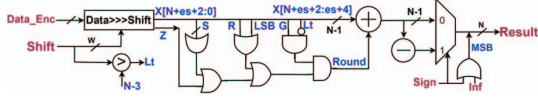


Fig. 4. Proposed architecture of posit encoder(b). R: $X[es+2]$, G: $X[es+3]$, LSB: $X[es+4]$, S: $X[es+1:0]$, Data_Enc: $\{Reg_Exp[es+1:0], Frac[N-1:0], 1'b0\}$

preamble Reg_Exp concatenated with fractional data is arithmetic right shifted by run length shift to get identical bits in posit data, rounded and converted to 2's complement format. In posit encoder(a), regime is extracted from OE and converted to unsigned format. Out of range values corresponding to regime values are mapped to $N - 1$. Run length shift is obtained by considering sign of the regime, overflow ($maxpos$), underflow ($minpos$) and special cases as shown in Fig. 3. Processing of special cases with overflow processing, eliminates N-bit multiplexer needed for handling special cases at the end. Followed rounding scheme is round to nearest even [2]. The information lost due to arithmetic right shifting in posit encoder(b) is preserved as Z and used for rounding. Rounding is disabled by Lt if the number is $maxpos$ or $minpos$.

D. Non-pipelined Posit Multiplier

We explored ways to reduce the datapath delay of existing posit multiplier designs [4] [6] [7] [9]. Proposed decoder design with unsigned fractional part as output in Section III.A is chosen. Fractional part multiplication is done using DSP48E blocks in Virtex 7 FPGA. In parallel to multiplication of fractional parts, processing of sign, special cases and OE addition ($OE_1 + OE_2$) can be done. For completing posit encoding(a), OE_3 ($OE_1 + OE_2 + \text{normalization value}$) is required. Normalisation value will be available once the multiplication is completed. The possible values of normalization values are 0 or 1. Then, posit encoding(a) and encoding(b) can be performed. This increases the datapath delay. To reduce datapath delay, posit encoder(a) block is repeated twice. One posit encoder(a) block takes normalisation value of 0 and another posit encoder(a) block takes normalisation value of 1. Later the correct value of Reg_Exp and Shift are chosen with normalisation value available after multiplication. In Parallel to selection, normalisation of fractional part F_3 is done. Thus, the datapath delay of posit encoder(a) is eliminated. The extra bits generated during multiplication are preserved in rounding.

E. Pipelined Posit Multiplier

Pipelined posit multiplier consists of nine stages. There are three stages for decoding, three stages for multiplication

(pipelined), and three stages for encoding. In first stage of decoding, identical bits are counted. The second and third stages are used for removing identical bits and termination bit, corrections in count of LIC, extracting Exp part and fractional part and regime computation. Multiplication is done in three stages along with addition of OE1 and OE2 and posit encoding(a). Normalisation and run length encoding is done in two stages. Rounding followed by 2's complement format conversion is done in the final stage of encoding. As es value varies, fractional data width ($N-es-2$) of multiplier and multiplicand changes. A methodology is proposed to balance delays in different stages for getting optimum clock frequency as shown in Algorithm 1. The steps 4 to 7 are done in first stage. The step 8 in algorithm is done in the second stage and steps 9 and 10 are done in the third stage of multiplication.

Algorithm 1: Balanced delay pipelined multiplier

1. $L \leftarrow N - es - 2$, $C \leftarrow \text{floor}(L/2)$, $M \leftarrow \text{modulo}(L, 2)$
2. **Input:** $F_1[L-1:0]$, $F_2[L-1:0]$; $G \leftarrow C + M$
3. **Output:** $F_3[2L-1:0]$
4. $W[2G-1:0] \leftarrow F_1[G-1:0] \times F_2[G-1:0]$
5. $X[L-1:0] \leftarrow F_1[L-1:G] \times F_2[G-1:0]$
6. $Y[L-1:0] \leftarrow F_1[G-1:0] \times F_2[L-1:G]$
7. $Z[2L-2G-1:0] \leftarrow F_1[L-1:G] \times F_2[L-1:G]$
8. $A[L:0] \leftarrow \{1'b0, X\} + \{1'b0, Y\}$; $W_1 \leftarrow W$; $Z_1 \leftarrow Z$;
9. $F_3[2L-1:G] \leftarrow A + W_1[2G-1:G]$;
10. $F_3[G-1:0] \leftarrow W_1[G-1:0]$

F. Non-pipelined Posit Adder

Proposed decoder design with 2's complement fractional part as output in Section III.B is chosen. After decoding, magnitude comparison of both OE's is done and sorting of fractional parts is performed. Lower magnitude operand's fractional part is shifted right (arithmetic) by difference of overall exponents (OE) for aligning with greater magnitude fractional part. Before aligning, both fractional parts are appended with zeros on LSB side. Appending three 0's on LSB side for lower magnitude fractional part will help in preserving the information of the bits lost during right shifting. Then, addition of both fractional parts is performed. Result in 2's complement format is converted to unsigned form. The unsigned form result is given to leading zero counter(LZC) for finding the normalisation value. Then the OE_{max} and normalisation value (can be positive or negative) are added to form OE_8 . Posit encoding(a) is done in parallel to left shifting of unsigned fractional part depending on normalisation value. Finally posit encoding(b) is performed to obtain the result. There is 100% accuracy of results in proposed posit adder, whereas PACoGen [4] gives an accuracy of around 95%.

G. Pipelined Posit Adder

Pipelined posit adder consists of ten stages. The block schematic is shown in Fig. 5. There are three stages for decoding, one stage for comparing and sorting of fractional

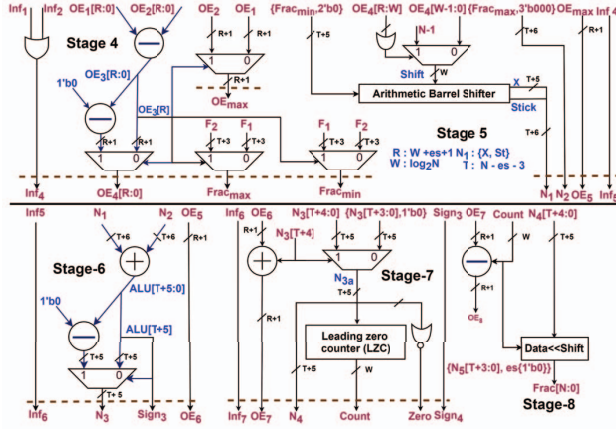


Fig. 5. Architecture of posit adder arithmetic operations

inputs and OE's, one stage for aligning lower fractional part, one stage for adding fractional parts and converting result to unsigned form, one stage counting the number of leading zero bits, one stage for normalisation and posit encoding(a) and two stages for posit encoding(b).

IV. EVALUATION

Generated bit stream of designed operators is configured on FPGA. Input test vectors along with its results from SoftPosit [10] are stored in Block ROM. The obtained result from designed operators are compared with results of SoftPosit. There is 100% match in results. SoftPosit provides support for $\langle 8,0 \rangle$, $\langle 16,1 \rangle$, $\langle 32,2 \rangle$. In order to test remaining $\langle N,es \rangle$ posit operators, software implementation of posit operators is done in MATLAB. Posit data inputs are converted to decimal and are again converted back to posit after arithmetic operation. Hardware and software results are matching 100% for all $\langle N,es \rangle$ possible combinations.

V. IMPLEMENTATION RESULTS

Implementation results of existing designs [4] [7] [9] corresponding to posit multiplier and posit adder (non-pipelined and pipelined) are reported on Xilinx Virtex-7 (xc7vx330t-3ffg1157) FPGA. Proposed posit multiplier and adder results are also reported on Virtex-7 FPGA.

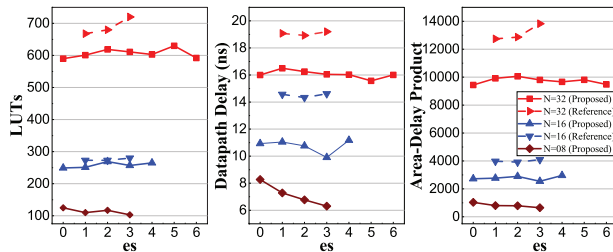


Fig. 6. Comparison of proposed non-pipelined $\langle N,es \rangle$ posit multiplier and reference [4] design in terms of LUTs, datapath delay and area delay product

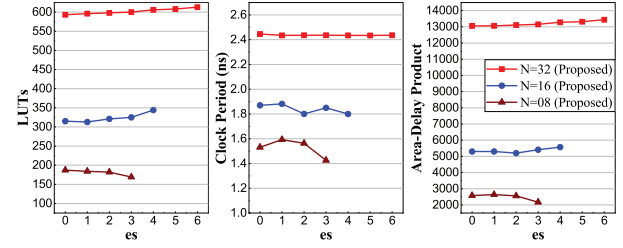


Fig. 7. LUTs, clock period and area delay product of proposed pipelined $\langle N,es \rangle$ posit multiplier

Non-pipelined posit multiplier results are shown in Fig. 6. Datapath delay has been reduced by 27% and number of LUTs are reduced by 6% compared to reference [4] for $\langle 16,es \rangle$. Datapath delay has been reduced by 15% and number of LUTs are reduced by 12% compared to reference [4] for $\langle 32,es \rangle$. Improvements in decoder and encoder designs has reduced datapath delay and LUTs. Four number of DSP48E blocks and one DSP48E block is used for $\langle 32,es \rangle$ and $\langle 16,es \rangle$ posit multiplier respectively. The proposed pipelined posit multiplier results along with its comparison with existing designs are shown in Fig. 7 and Table I.

TABLE I
PIPELINED POSIT MULTIPLIER COMPARISON WITH EXISTING DESIGNS

Designs	$\langle N,es \rangle$	LUTs	Period (ns)	Latency
IP Core [7]	$\langle 32,3 \rangle$	1303	4.97	11
Proposed	$\langle 32,3 \rangle$	600 (53% ↓)	2.44 (50% ↓)	9
PACoGen [4]	$\langle 32,6 \rangle$	854	4.4	6
Proposed	$\langle 32,6 \rangle$	613 (28% ↓)	2.44 (44% ↓)	9

Non-pipelined posit adder results are shown in Fig. 8. Number of LUTs are reduced by 9% and datapath delay has been reduced by 7% compared to reference design [4] for $\langle 16,es \rangle$. The reduction in LUTs is because of decoder's output fractional part which is in 2's complement form favorable for additions. Number of LUTs are reduced by 22% and datapath delay has not been reduced much compared to reference design [4] for $\langle 32,es \rangle$. The proposed pipelined posit adder results along with its comparison with existing designs are shown in Fig. 9 and Table II.

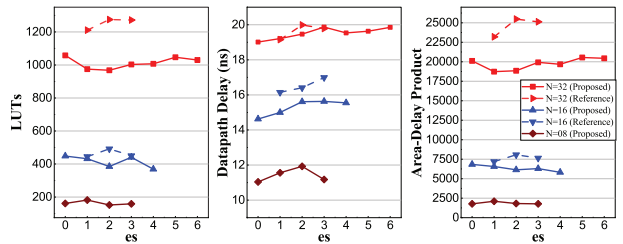


Fig. 8. Comparison of proposed non-pipelined $\langle N,es \rangle$ posit adder and reference [4] design in terms of LUTs, datapath delay and area delay product

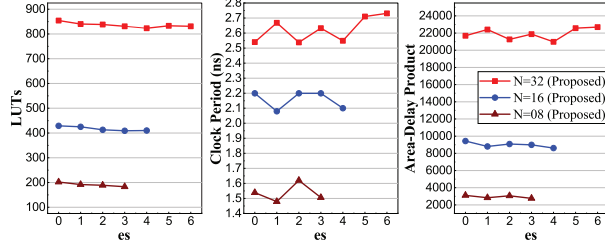


Fig. 9. LUTs, clock period and area delay product of proposed pipelined $\langle N, es \rangle$ posit adder

TABLE II
PIPELINED POSIT ADDERS COMPARISON WITH EXISTING DESIGNS

Designs	$\langle N, es \rangle$	LUTs	Period (ns)	Latency
IP Core [7]	$\langle 32, 3 \rangle$	1053	4.92	12
Proposed	$\langle 32, 3 \rangle$	831 (21% ↓)	2.74	10
PACoGen [4]	$\langle 32, 6 \rangle$	946	4.1	5
Proposed	$\langle 32, 6 \rangle$	831 (12% ↓)	2.75	10

VI. CONCLUSIONS AND FUTURE WORK

In this paper, hardware implementation of decoding and encoding of posit numbers has been done in an optimized way and used in posit multiplier and posit adder designs. The datapath delay, LUTs and area delay product of our proposed posit multiplier and adder designs (non-pipelined and pipelined) are better than existing designs [4] [6] [7] [9]. Future work includes implementation of posit division and posit square root using the proposed decoding and encoding techniques. Furthermore, posit arithmetic unit (PAU) can be done by integrating posit adder, posit multiplier, posit divider, posit square root and posit multiply accumulate unit (MAC).

REFERENCES

- [1] "IEEE standard for floating-point arithmetic," IEEE 754-2019, doi: 10.1109/IEEESTD.2019.8766229.
- [2] J. L. Gustafson and I. T. Yonemoto, "Beating floating point at its own game: Posit arithmetic," *Supercomputing Frontiers and Innovations*, vol. 4, no. 2, pp. 71–86, 2017.
- [3] M. K. Jaiswal and H. K.-H. So, "Architecture generator for type-3 unum posit adder/subtractor," in *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2018, pp. 1–5.
- [4] M. K. Jaiswal and H. K.-H. So, "Pacogen: A hardware posit arithmetic core generator," *IEEE Access*, vol. 7, pp. 74 586–74 601, 2019.
- [5] PACoGen: Posit Arithmetic Core Generator. Available: <https://github.com/manish-kj/PACoGen>
- [6] R. Chaurasiya, J. Gustafson, R. Shrestha, J. Neudorfer, S. Nambiar, K. Niyogi, F. Merchant, and R. Leupers, "Parameterized Posit Arithmetic Hardware Generator," in *36th International Conference on Computer Design*. IEEE, 2018, pp. 334–341.
- [7] J. Hou, Y. Zhu, S. Du, and S. Shong, "Enhancing accuracy and dynamic range of scientific data analytics by implementing posit arithmetic on FPGA," *J. Signal Process. Syst.*, pp. 1–12, Nov. 2018. doi: 10.1007/s11265-018-1420-5.
- [8] Yohann Uguen, Luc Forget, Florent de Dinechin, "Evaluating the hardware cost of the posit number system," in *29th International Conference on Field Programmable Logic & Applications (FPL)*, 2019, pp. 106–113.
- [9] Lakshmi Bhanuprakash Reddy K, Haripriya R S, Keerthija Puli, Subba Ramkumar Reddy Annappalli, Vikramkumar Pudi, "Design of Energy Efficient and Low Delay Posit Multiplier," *36th International Conference on VLSI Design and 2023 22nd Interna-*

tional Conference on Embedded Systems (VLSID), pp. 1–6, 2023.
doi.org/10.1109/VLSID57277.2023.00042
[10] <https://crates.io/crates/softposit>